

```
def f1():
    x = 88
    def f2(x=x):                # Remember enclosing scope X with defaults
        print(x)
    f2()

f1()                            # Prints 88
```

This code works in all Python releases, and you'll still see this pattern in some existing Python code. In short, the syntax `arg = val` in a `def` header means that the argument `arg` will default to the value `val` if no real value is passed to `arg` in a call.

In the modified `f2` here, the `x=x` means that the argument `x` will default to the value of `x` in the enclosing scope—because the second `x` is evaluated before Python steps into the nested `def`, it still refers to the `x` in `f1`. In effect, the default remembers what `x` was in `f1` (i.e., the object `88`).

That's fairly complex, and it depends entirely on the timing of default value evaluations. In fact, the nested scope lookup rule was added to Python to make defaults unnecessary for this role—today, Python automatically remembers any values required in the enclosing scope for use in nested `defs`.

Of course, the best prescription for most code is simply to avoid nesting `defs` within `defs`, as it will make your programs much simpler. The following is an equivalent of the prior example that banishes the notion of nesting. Notice the forward reference in this code—it's OK to call a function defined after the function that calls it, as long as the second `def` runs before the first function is actually called. Code inside a `def` is never evaluated until the function is actually called:

```
>>> def f1():
...     x = 88                # Pass x along instead of nesting
...     f2(x)                # Forward reference okay
...
>>> def f2(x):
...     print(x)
...
>>> f1()
88
```

If you avoid nesting this way, you can almost forget about the nested scopes concept in Python, unless you need to code in the factory function style discussed earlier—at least, for `def` statements. `lambdas`, which almost naturally appear nested in `defs`, often rely on nested scopes, as the next section explains.

Nested scopes and lambdas

While they're rarely used in practice for `defs` themselves, you are more likely to care about nested function scopes when you start coding `lambda` expressions. We won't cover `lambda` in depth until [Chapter 19](#), but in short, it's an expression that generates a new function to be called later, much like a `def` statement. Because it's an expression,

though, it can be used in places that `def` cannot, such as within list and dictionary literals.

Like a `def`, a `lambda` expression introduces a new local scope for the function it creates. Thanks to the enclosing scopes lookup layer, `lambdas` can see all the variables that live in the functions in which they are coded. Thus, the following code works, but only because the nested scope rules are applied:

```
def func():
    x = 4
    action = (lambda n: x ** n)      # x remembered from enclosing def
    return action

x = func()
print(x(2))                          # Prints 16, 4 ** 2
```

Prior to the introduction of nested function scopes, programmers used defaults to pass values from an enclosing scope into `lambdas`, just as for `defs`. For instance, the following works on all Python releases:

```
def func():
    x = 4
    action = (lambda n, x=x: x ** n)  # Pass x in manually
    return action
```

Because `lambdas` are expressions, they naturally (and even normally) nest inside enclosing `defs`. Hence, they are perhaps the biggest beneficiaries of the addition of enclosing function scopes in the lookup rules; in most cases, it is no longer necessary to pass values into `lambdas` with defaults.

Scopes versus defaults with loop variables

There is one notable exception to the rule I just gave: if a `lambda` or `def` defined within a function is nested inside a loop, and the nested function references an enclosing scope variable that is changed by that loop, all functions generated within the loop will have the same value—the value the referenced variable had in the last loop iteration.

For instance, the following attempts to build up a list of functions that each remember the current variable `i` from the enclosing scope:

```
>>> def makeActions():
...     acts = []
...     for i in range(5):
...         acts.append(lambda x: i ** x)      # Tries to remember each i
...         # All remember same last i!
...     return acts
...
>>> acts = makeActions()
>>> acts[0]
<function <lambda> at 0x012B16B0>
```

This doesn't quite work, though—because the enclosing scope variable is looked up when the nested functions are later *called*, they all effectively remember the same value

(the value the loop variable had on the *last* loop iteration). That is, we get back 4 to the power of 2 for each function in the list, because *i* is the same in all of them:

```
>>> acts[0](2)           # All are 4 ** 2, value of last i
16
>>> acts[2](2)           # This should be 2 ** 2
16
>>> acts[4](2)           # This should be 4 ** 2
16
```

This is the one case where we still have to explicitly retain enclosing scope values with default arguments, rather than enclosing scope references. That is, to make this sort of code work, we must pass in the *current* value of the enclosing scope's variable with a default. Because defaults are evaluated when the nested function is *created* (not when it's later *called*), each remembers its own value for *i*:

```
>>> def makeActions():
...     acts = []
...     for i in range(5):           # Use defaults instead
...         acts.append(lambda x, i=i: i ** x)  # Remember current i
...     return acts
...
>>> acts = makeActions()
>>> acts[0](2)                   # 0 ** 2
0
>>> acts[2](2)                   # 2 ** 2
4
>>> acts[4](2)                   # 4 ** 2
16
```

This is a fairly obscure case, but it can come up in practice, especially in code that generates callback handler functions for a number of widgets in a GUI (e.g., button-press handlers). We'll talk more about defaults in [Chapter 18](#) and *lambdas* in [Chapter 19](#), so you may want to return and review this section later.‡

Arbitrary scope nesting

Before ending this discussion, I should note that scopes may nest arbitrarily, but only enclosing function *def* statements (not classes, described in [Part VI](#)) are searched:

```
>>> def f1():
...     x = 99
...     def f2():
...         def f3():
...             print(x)           # Found in f1's local scope!
...         f3()
```

‡ In the section “[Function Gotchas](#)” on [page 518](#) at the end of this part of the book, we'll also see that there is an issue with using mutable objects like lists and dictionaries for default arguments (e.g., `def f(a=[])`)—because defaults are implemented as single objects attached to functions, mutable defaults retain state from call to call, rather than being initialized anew on each call. Depending on whom you ask, this is either considered a feature that supports state retention, or a strange wart on the language. More on this at the end of [Chapter 20](#).

```
...     f2()
...
>>> f1()
99
```

Python will search the local scopes of *all* enclosing `defs`, from inner to outer, after the referencing function's local scope and before the module's global scope or built-ins. However, this sort of code is even less likely to pop up in practice. In Python, we say *flat is better than nested*—except in very limited contexts, your life (and the lives of your coworkers) will generally be better if you minimize nested function definitions.

The nonlocal Statement

In the prior section we explored the way that nested functions can *reference* variables in an enclosing function's scope, even if that function has already returned. It turns out that, as of Python 3.0, we can also *change* such enclosing scope variables, as long as we declare them in `nonlocal` statements. With this statement, nested `defs` can have both read and write access to names in enclosing functions.

The `nonlocal` statement is a close cousin to `global`, covered earlier. Like `global`, `nonlocal` declares that a name will be changed in an enclosing scope. Unlike `global`, though, `nonlocal` applies to a name in an enclosing function's scope, not the global module scope outside all `defs`. Also unlike `global`, `nonlocal` names must already exist in the enclosing function's scope when declared—they can exist only in enclosing functions and cannot be created by a first assignment in a nested `def`.

In other words, `nonlocal` both allows assignment to names in enclosing function scopes and limits scope lookups for such names to enclosing `defs`. The net effect is a more direct and reliable implementation of changeable scope information, for programs that do not desire or need classes with attributes.

nonlocal Basics

Python 3.0 introduces a new `nonlocal` statement, which has meaning only inside a function:

```
def func():
    nonlocal name1, name2, ...
```

This statement allows a nested function to change one or more names defined in a syntactically enclosing function's scope. In Python 2.X (including 2.6), when one function `def` is nested in another, the nested function can reference any of the names defined by assignment in the enclosing `def`'s scope, but it cannot change them. In 3.0, declaring the enclosing scopes' names in a `nonlocal` statement enables nested functions to assign and thus change such names as well.

This provides a way for enclosing functions to provide *writable* state information, remembered when the nested function is later called. Allowing the state to change

makes it more useful to the nested function (imagine a counter in the enclosing scope, for instance). In 2.X, programmers usually achieve similar goals by using classes or other schemes. Because nested functions have become a more common coding pattern for state retention, though, `nonlocal` makes it more generally applicable.

Besides allowing names in enclosing `defs` to be changed, the `nonlocal` statement also forces the issue for references—just like the `global` statement, `nonlocal` causes searches for the names listed in the statement to begin in the enclosing `defs`’ scopes, not in the local scope of the declaring function. That is, `nonlocal` also means “skip my local scope entirely.”

In fact, the names listed in a `nonlocal` *must* have been previously defined in an enclosing `def` when the `nonlocal` is reached, or an error is raised. The net effect is much like `global`: `global` means the names reside in the enclosing module, and `nonlocal` means they reside in an enclosing `def`. `nonlocal` is even more strict, though—scope search is restricted to *only* enclosing `defs`. That is, `nonlocal` names can appear only in enclosing `defs`, not in the module’s global scope or built-in scopes outside the `defs`.

The addition of `nonlocal` does not alter name reference scope rules in general; they still work as before, per the “LEGB” rule described earlier. The `nonlocal` statement mostly serves to allow names in enclosing scopes to be changed rather than just referenced. However, `global` and `nonlocal` statements do both restrict the lookup rules somewhat, when coded in a function:

- `global` makes scope lookup begin in the enclosing module’s scope and allows names there to be assigned. Scope lookup continues on to the built-in scope if the name does not exist in the module, but assignments to global names always create or change them in the module’s scope.
- `nonlocal` restricts scope lookup to just enclosing `defs`, requires that the names already exist there, and allows them to be assigned. Scope lookup does not continue on to the global or built-in scopes.

In Python 2.6, references to enclosing `def` scope names are allowed, but not assignment. However, you can still use classes with explicit attributes to achieve the same changeable state information effect as nonlocals (and you may be better off doing so in some contexts); globals and function attributes can sometimes accomplish similar goals as well. More on this in a moment; first, let’s turn to some working code to make this more concrete.

nonlocal in Action

On to some examples, all run in 3.0. References to enclosing `def` scopes work as they do in 2.6. In the following, `tester` builds and returns the function `nested`, to be called later, and the `state` reference in `nested` maps the local scope of `tester` using the normal scope lookup rules:

```
C:\misc>c:\python30\python
```

```
>>> def tester(start):
...     state = start           # Referencing nonlocals works normally
...     def nested(label):
...         print(label, state) # Remembers state in enclosing scope
...         return nested
...
>>> F = tester(0)
>>> F('spam')
spam 0
>>> F('ham')
ham 0
```

Changing a name in an enclosing `def`'s scope is not allowed by default, though; this is the normal case in 2.6 as well:

```
>>> def tester(start):
...     state = start
...     def nested(label):
...         print(label, state)
...         state += 1         # Cannot change by default (or in 2.6)
...         return nested
...
>>> F = tester(0)
>>> F('spam')
UnboundLocalError: local variable 'state' referenced before assignment
```

Using `nonlocal` for changes

Now, under 3.0, if we declare `state` in the `tester` scope as `nonlocal` within `nested`, we get to change it inside the nested function, too. This works even though `tester` has returned and exited by the time we call the returned `nested` function through the name `F`:

```
>>> def tester(start):
...     state = start           # Each call gets its own state
...     def nested(label):
...         nonlocal state      # Remembers state in enclosing scope
...         print(label, state)
...         state += 1         # Allowed to change it if nonlocal
...         return nested
...
>>> F = tester(0)
>>> F('spam')                 # Increments state on each call
spam 0
>>> F('ham')
ham 1
>>> F('eggs')
eggs 2
```

As usual with enclosing scope references, we can call the `tester` factory function multiple times to get multiple copies of its state in memory. The `state` object in the enclosing scope is essentially attached to the `nested` function object returned; each call makes a

new, distinct state object, such that updating one function's state won't impact the other. The following continues the prior listing's interaction:

```
>>> G = tester(42)           # Make a new tester that starts at 42
>>> G('spam')
spam 42

>>> G('eggs')                # My state information updated to 43
eggs 43

>>> F('bacon')               # But F's is where it left off: at 3
bacon 3                       # Each call has different state information
```

Boundary cases

There are a few things to watch out for. First, unlike the `global` statement, `nonlocal` names really *must* have previously been assigned in an enclosing `def`'s scope when a `nonlocal` is evaluated, or else you'll get an error—you cannot create them dynamically by assigning them anew in the enclosing scope:

```
>>> def tester(start):
...     def nested(label):
...         nonlocal state      # Nonlocals must already exist in enclosing def!
...         state = 0
...         print(label, state)
...     return nested
...
SyntaxError: no binding for nonlocal 'state' found

>>> def tester(start):
...     def nested(label):
...         global state        # Globals don't have to exist yet when declared
...         state = 0          # This creates the name in the module now
...         print(label, state)
...     return nested
...
>>> F = tester(0)
>>> F('abc')
abc 0
>>> state
0
```

Second, `nonlocal` restricts the scope lookup to just enclosing `defs`; nonlocals are not looked up in the enclosing module's global scope or the built-in scope outside all `defs`, even if they are already there:

```
>>> spam = 99
>>> def tester():
...     def nested():
...         nonlocal spam      # Must be in a def, not the module!
...         print('Current=', spam)
...         spam += 1
...     return nested
...
SyntaxError: no binding for nonlocal 'spam' found
```

These restrictions make sense once you realize that Python would not otherwise generally know which enclosing scope to create a brand new name in. In the prior listing, should `spam` be assigned in `tester`, or the module outside? Because this is ambiguous, Python must resolve nonlocals at function *creation* time, not function *call* time.

Why nonlocal?

Given the extra complexity of nested functions, you might wonder what the fuss is about. Although it's difficult to see in our small examples, state information becomes crucial in many programs. There are a variety of ways to “remember” information across function and method calls in Python. While there are tradeoffs for all, `nonlocal` does improve this story for enclosing scope references—the `nonlocal` statement allows multiple copies of changeable state to be retained in memory and addresses simple state-retention needs where classes may not be warranted.

As we saw in the prior section, the following code allows state to be retained and modified in an enclosing scope. Each call to `tester` creates a little self-contained *package of changeable information*, whose names do not clash with any other part of the program:

```
def tester(start):
    state = start                                # Each call gets its own state
    def nested(label):
        nonlocal state                          # Remembers state in enclosing scope
        print(label, state)
        state += 1                              # Allowed to change it if nonlocal
    return nested

F = tester(0)
F('spam')
```

Unfortunately, this code only works in Python 3.0. If you are using Python 2.6, other options are available, depending on your goals. The next two sections present some alternatives.

Shared state with globals

One usual prescription for achieving the `nonlocal` effect in 2.6 and earlier is to simply move the state out to the *global scope* (the enclosing module):

```
>>> def tester(start):
...     global state                            # Move it out to the module to change it
...     state = start                          # global allows changes in module scope
...     def nested(label):
...         global state
...         print(label, state)
...         state += 1
...     return nested
...
>>> F = tester(0)
>>> F('spam')                                # Each call increments shared global state
```

```
spam 0
>>> F('eggs')
eggs 1
```

This works in this case, but it requires `global` declarations in both functions and is prone to name collisions in the global scope (what if “state” is already being used?). A worse, and more subtle, problem is that it only allows for a *single shared copy* of the state information in the module scope—if we call `tester` again, we’ll wind up resetting the module’s state variable, such that prior calls will see their state overwritten:

```
>>> G = tester(42)                                # Resets state's single copy in global scope
>>> G('toast')
toast 42

>>> G('bacon')
bacon 43

>>> F('ham')                                       # Oops -- my counter has been overwritten!
ham 44
```

As shown earlier, when using `nonlocal` instead of `global`, each call to `tester` remembers its own unique copy of the state object.

State with classes (preview)

The other prescription for changeable state information in 2.6 and earlier is to use *classes with attributes* to make state information access more explicit than the implicit magic of scope lookup rules. As an added benefit, each instance of a class gets a fresh copy of the state information, as a natural byproduct of Python’s object model.

We haven’t explored classes in detail yet, but as a brief preview, here is a reformulation of the `tester/nested` functions used earlier as a class—state is recorded in objects explicitly as they are created. To make sense of this code, you need to know that a `def` within a `class` like this works exactly like a `def` outside of a `class`, except that the function’s `self` argument automatically receives the implied subject of the call (an instance object created by calling the class itself):

```
>>> class tester:                                # Class-based alternative (see Part VI)
...     def __init__(self, start):                # On object construction,
...         self.state = start                    # save state explicitly in new object
...     def nested(self, label):
...         print(label, self.state)              # Reference state explicitly
...         self.state += 1                       # Changes are always allowed
...
>>> F = tester(0)                                # Create instance, invoke __init__
>>> F.nested('spam')                             # F is passed to self
spam 0
>>> F.nested('ham')
ham 1

>>> G = tester(42)                                # Each instance gets new copy of state
>>> G.nested('toast')                             # Changing one does not impact others
toast 42
```

```

>>> G.nested('bacon')
bacon 43

>>> F.nested('eggs')           # F's state is where it left off
eggs 2
>>> F.state                     # State may be accessed outside class
3

```

With just slightly more magic, which we'll delve into later in this book, we could also make our class look like a callable function using operator overloading. `__call__` intercepts direct calls on an instance, so we don't need to call a named method:

```

>>> class tester:
...     def __init__(self, start):
...         self.state = start
...     def __call__(self, label):           # Intercept direct instance calls
...         print(label, self.state)        # So .nested() not required
...         self.state += 1
...
>>> H = tester(99)
>>> H('juice')                           # Invokes __call__
juice 99
>>> H('pancakes')
pancakes 100

```

Don't sweat the details in this code too much at this point in the book; we'll explore classes in depth in [Part VI](#) and will look at specific operator overloading tools like `__call__` in [Chapter 29](#), so you may wish to file this code away for future reference. The point here is that classes can make state information more obvious, by leveraging explicit attribute assignment instead of scope lookups.

While using classes for state information is generally a good rule of thumb to follow, they might be overkill in cases like this, where state is a single counter. Such trivial state cases are more common than you might think; in such contexts, nested `defs` are sometimes more lightweight than coding classes, especially if you're not familiar with OOP yet. Moreover, there are some scenarios in which nested `defs` may actually work better than classes (see the description of *method decorators* in [Chapter 38](#) for an example that is far beyond this chapter's scope).

State with function attributes

As a final state-retention option, we can also sometimes achieve the same effect as nonlocals with *function attributes*—user-defined names attached to functions directly. Here's a final version of our example based on this technique—it replaces a nonlocal with an attribute attached to the nested function. Although this scheme may not be as intuitive to some, it also allows the state variable to be accessed *outside* the nested function (with nonlocals, we can only see state variables within the nested `def`):

```

>>> def tester(start):
...     def nested(label):
...         print(label, nested.state)      # nested is in enclosing scope
...         nested.state += 1               # Change attr, not nested itself

```

```

...     nested.state = start           # Initial state after func defined
...     return nested
...
>>> F = tester(0)
>>> F('spam')                        # F is a 'nested' with state attached
spam 0
>>> F('ham')
ham 1
>>> F.state                           # Can access state outside functions too
2
>>>
>>> G = tester(42)                   # G has own state, doesn't overwrite F's
>>> G('eggs')
eggs 42
>>> F('ham')
ham 2

```

This code relies on the fact that the function name `nested` is a local variable in the `tester` scope enclosing `nested`; as such, it can be referenced freely inside `nested`. This code also relies on the fact that changing an object in-place is not an assignment to a name; when it increments `nested.state`, it is changing part of the object `nested` references, not the name `nested` itself. Because we're not really assigning a name in the enclosing scope, no `nonlocal` is needed.

As you can see, globals, nonlocals, classes, and function attributes all offer state-retention options. Globals only support shared data, classes require a basic knowledge of OOP, and both classes and function attributes allow state to be accessed outside the nested function itself. As usual, the best tool for your program depends upon your program's goals.

Chapter Summary

In this chapter, we studied one of two key concepts related to functions: *scopes* (how variables are looked up when they are used). As we learned, variables are considered local to the function definitions in which they are assigned, unless they are specifically declared to be global or nonlocal. We also studied some more advanced scope concepts here, including nested function scopes and function attributes. Finally, we looked at some general design ideas, such as the need to avoid globals and cross-file changes.

In the next chapter, we're going to continue our function tour with the second key function-related concept: argument passing. As we'll find, arguments are passed into a function by assignment, but Python also provides tools that allow functions to be flexible in how items are passed. Before we move on, let's take this chapter's quiz to review the scope concepts we've covered here.

Test Your Knowledge: Quiz

1. What is the output of the following code, and why?

```
>>> X = 'Spam'
>>> def func():
...     print(X)
...
>>> func()
```

2. What is the output of this code, and why?

```
>>> X = 'Spam'
>>> def func():
...     X = 'NI!'
...
>>> func()
>>> print(X)
```

3. What does this code print, and why?

```
>>> X = 'Spam'
>>> def func():
...     X = 'NI'
...     print(X)
...
>>> func()
>>> print(X)
```

4. What output does this code produce? Why?

```
>>> X = 'Spam'
>>> def func():
...     global X
...     X = 'NI'
...
>>> func()
>>> print(X)
```

5. What about this code—what's the output, and why?

```
>>> X = 'Spam'
>>> def func():
...     X = 'NI'
...     def nested():
...         print(X)
...     nested()
...
>>> func()
>>> X
```

6. How about this example: what is its output in Python 3.0, and why?

```
>>> def func():
...     X = 'NI'
...     def nested():
...         nonlocal X
...         X = 'Spam'
...     nested()
...     print(X)
...
>>> func()
```

7. Name three or more ways to retain state information in a Python function.

Test Your Knowledge: Answers

1. The output here is 'Spam', because the function references a global variable in the enclosing module (because it is not assigned in the function, it is considered global).
2. The output here is 'Spam' again because assigning the variable inside the function makes it a local and effectively hides the global of the same name. The `print` statement finds the variable unchanged in the global (module) scope.
3. It prints 'NI' on one line and 'Spam' on another, because the reference to the variable within the function finds the assigned local and the reference in the `print` statement finds the global.
4. This time it just prints 'NI' because the global declaration forces the variable assigned inside the function to refer to the variable in the enclosing global scope.
5. The output in this case is again 'NI' on one line and 'Spam' on another, because the `print` statement in the nested function finds the name in the enclosing function's local scope, and the `print` at the end finds the variable in the global scope.
6. This example prints 'Spam', because the `nonlocal` statement (available in Python 3.0 but not 2.6) means that the assignment to `X` inside the nested function changes `X` in the enclosing function's local scope. Without this statement, this assignment would classify `X` as local to the nested function, making it a different variable; the code would then print 'NI' instead.
7. Although the values of local variables go away when a function returns, you can make a Python function retain state information by using shared global variables, enclosing function scope references within nested functions, or using default argument values. Function attributes can sometimes allow state to be attached to the function itself, instead of looked up in scopes. Another alternative, using OOP with classes, sometimes supports state retention better than any of the scope-based techniques because it makes it explicit with attribute assignments; we'll explore this option in [Part VI](#).

Arguments

[Chapter 17](#) explored the details behind Python’s *scopes*—the places where variables are defined and looked up. As we learned, the place where a name is defined in our code determines much of its meaning. This chapter continues the function story by studying the concepts in Python *argument passing*—the way that objects are sent to functions as inputs. As we’ll see, arguments (a.k.a. parameters) are assigned to names in a function, but they have more to do with object references than with variable scopes. We’ll also find that Python provides extra tools, such as keywords, defaults, and arbitrary argument collectors, that allow for wide flexibility in the way arguments are sent to a function.

Argument-Passing Basics

Earlier in this part of the book, I noted that arguments are passed by *assignment*. This has a few ramifications that aren’t always obvious to beginners, which I’ll expand on in this section. Here is a rundown of the key points in passing arguments to functions:

- **Arguments are passed by automatically assigning objects to local variable names.** Function arguments—references to (possibly) shared objects sent by the caller—are just another instance of Python assignment at work. Because references are implemented as pointers, all arguments are, in effect, passed by pointer. Objects passed as arguments are never automatically copied.
- **Assigning to argument names inside a function does not affect the caller.** Argument names in the function header become new, local names when the function runs, in the scope of the function. There is no aliasing between function argument names and variable names in the scope of the caller.
- **Changing a mutable object argument in a function may impact the caller.** On the other hand, as arguments are simply assigned to passed-in objects, functions can change passed-in mutable objects in place, and the results may affect the caller. Mutable arguments can be input and output for functions.

For more details on *references*, see [Chapter 6](#); everything we learned there also applies to function arguments, though the assignment to argument names is automatic and implicit.

Python’s pass-by-assignment scheme isn’t quite the same as C++’s reference parameters option, but it turns out to be very similar to the C language’s argument-passing model in practice:

- **Immutable arguments are effectively passed “by value.”** Objects such as integers and strings are passed by object reference instead of by copying, but because you can’t change immutable objects in-place anyhow, the effect is much like making a copy.
- **Mutable arguments are effectively passed “by pointer.”** Objects such as lists and dictionaries are also passed by object reference, which is similar to the way C passes arrays as pointers—mutable objects can be changed in-place in the function, much like C arrays.

Of course, if you’ve never used C, Python’s argument-passing mode will seem simpler still—it involves just the assignment of objects to names, and it works the same whether the objects are mutable or not.

Arguments and Shared References

To illustrate argument-passing properties at work, consider the following code:

```
>>> def f(a):                # a is assigned to (references) passed object
...     a = 99                # Changes local variable a only
...
>>> b = 88
>>> f(b)                     # a and b both reference same 88 initially
>>> print(b)                 # b is not changed
88
```

In this example the variable `a` is assigned the object `88` at the moment the function is called with `f(b)`, but `a` lives only within the called function. Changing `a` inside the function has no effect on the place where the function is called; it simply resets the local variable `a` to a completely different object.

That’s what is meant by a lack of name *aliasing*—assignment to an argument name inside a function (e.g., `a=99`) does not magically change a variable like `b` in the scope of the function call. Argument names may share passed objects initially (they are essentially pointers to those objects), but only temporarily, when the function is first called. As soon as an argument name is reassigned, this relationship ends.

At least, that’s the case for assignment to argument *names* themselves. When arguments are passed *mutable* objects like lists and dictionaries, we also need to be aware that in-place changes to such *objects* may live on after a function exits, and hence impact callers. Here’s an example that demonstrates this behavior:

```

>>> def changer(a, b):      # Arguments assigned references to objects
...     a = 2               # Changes local name's value only
...     b[0] = 'spam'       # Changes shared object in-place
...
>>> X = 1
>>> L = [1, 2]              # Caller
>>> changer(X, L)           # Pass immutable and mutable objects
>>> X, L                    # X is unchanged, L is different!
(1, ['spam', 2])

```

In this code, the `changer` function assigns values to argument `a` itself, and to a component of the object referenced by argument `b`. These two assignments within the function are only slightly different in syntax but have radically different results:

- Because `a` is a local variable name in the function's scope, the first assignment has no effect on the caller—it simply changes the local variable `a` to reference a completely different object, and does not change the binding of the name `X` in the caller's scope. This is the same as in the prior example.
- Argument `b` is a local variable name, too, but it is passed a mutable object (the list that `L` references in the caller's scope). As the second assignment is an in-place object change, the result of the assignment to `b[0]` in the function impacts the value of `L` after the function returns.

Really, the second assignment statement in `changer` doesn't change `b`—it changes part of the object that `b` currently references. This in-place change impacts the caller only because the changed object outlives the function call. The name `L` hasn't changed either—it still references the same, changed object—but it seems as though `L` differs after the call because the value it references has been modified within the function.

Figure 18-1 illustrates the name/object bindings that exist immediately after the function has been called, and before its code has run.

If this example is still confusing, it may help to notice that the effect of the automatic assignments of the passed-in arguments is the same as running a series of simple assignment statements. In terms of the first argument, the assignment has no effect on the caller:

```

>>> X = 1
>>> a = X                # They share the same object
>>> a = 2                # Resets 'a' only, 'X' is still 1
>>> print(X)
1

```

The assignment through the second argument does affect a variable at the call, though, because it is an in-place object change:

```

>>> L = [1, 2]
>>> b = L                # They share the same object
>>> b[0] = 'spam'        # In-place change: 'L' sees the change too
>>> print(L)
['spam', 2]

```

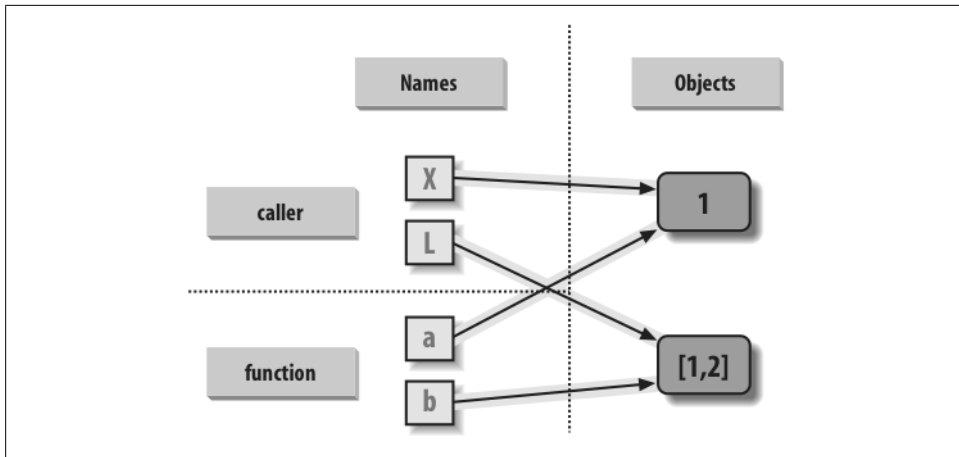


Figure 18-1. *References: arguments.* Because arguments are passed by assignment, argument names in the function may share objects with variables in the scope of the call. Hence, in-place changes to mutable arguments in a function can impact the caller. Here, *a* and *b* in the function initially reference the objects referenced by variables *X* and *L* when the function is first called. Changing the list through variable *b* makes *L* appear different after the call returns.

If you recall our discussions about shared mutable objects in Chapters 6 and 9, you'll recognize the phenomenon at work: changing a mutable object in-place can impact other references to that object. Here, the effect is to make one of the arguments work like both an input and an *output* of the function.

Avoiding Mutable Argument Changes

This behavior of in-place changes to mutable arguments isn't a bug—it's simply the way argument passing works in Python. Arguments are passed to functions by reference (a.k.a. pointer) by default because that is what we normally want. It means we can pass large objects around our programs without making multiple copies along the way, and we can easily update these objects as we go. In fact, as we'll see in [Part VI](#), Python's class model *depends* upon changing a passed-in “self” argument in-place, to update object state.

If we don't want in-place changes within functions to impact objects we pass to them, though, we can simply make explicit copies of mutable objects, as we learned in [Chapter 6](#). For function arguments, we can always copy the list at the point of call:

```
L = [1, 2]
changer(X, L[:])    # Pass a copy, so our 'L' does not change
```

We can also copy within the function itself, if we never want to change passed-in objects, regardless of how the function is called:

```
def changer(a, b):
    b = b[:]        # Copy input list so we don't impact caller
```

```
a = 2
b[0] = 'spam'      # Changes our list copy only
```

Both of these copying schemes don't stop the function from changing the object—they just prevent those changes from impacting the caller. To really prevent changes, we can always convert to immutable objects to force the issue. Tuples, for example, throw an exception when changes are attempted:

```
L = [1, 2]
changer(X, tuple(L))    # Pass a tuple, so changes are errors
```

This scheme uses the built-in `tuple` function, which builds a new tuple out of all the items in a sequence (really, any iterable). It's also something of an extreme—because it forces the function to be written to never change passed-in arguments, this solution might impose more limitations on the function than it should, and so should generally be avoided (you never know when changing arguments might come in handy for other calls in the future). Using this technique will also make the function lose the ability to call any list-specific methods on the argument, including methods that do not change the object in-place.

The main point to remember here is that functions might update mutable objects like lists and dictionaries passed into them. This isn't necessarily a problem if it's expected, and often serves useful purposes. Moreover, functions that change passed-in mutable objects in place are probably designed and intended to do so—the change is likely part of a well-defined API that you shouldn't violate by making copies.

However, you do have to be aware of this property—if objects change out from under you unexpectedly, check whether a called function might be responsible, and make copies when objects are passed if needed.

Simulating Output Parameters

We've already discussed the `return` statement and used it in a few examples. Here's another way to use this statement: because `return` can send back any sort of object, it can return *multiple values* by packaging them in a tuple or other collection type. In fact, although Python doesn't support what some languages label “call-by-reference” argument passing, we can usually simulate it by returning tuples and assigning the results back to the original argument names in the caller:

```
>>> def multiple(x, y):
...     x = 2          # Changes local names only
...     y = [3, 4]
...     return x, y    # Return new values in a tuple
...
>>> X = 1
>>> L = [1, 2]
>>> X, L = multiple(X, L)  # Assign results to caller's names
>>> X, L
(2, [3, 4])
```